

Workgroup: FAPI
Published: 17 August 2025
Status: Final
Authors: T. Lodderstedt B. Campbell
SPRIND Ping Identity

JWT Secured Authorization Response Mode for OAuth 2.0 (JARM) incorporating errata set 1

Table of Contents

1. Introduction
 - 1.1. Notational Conventions
 - 1.2. Terms and definitions
 - 1.3. Symbols and Abbreviated terms
2. JWT-based Response Mode
 - 2.1. The JWT Response Document
 - 2.1.1. Example Response Type "code"
 - 2.2. Signing and Encryption
 - 2.3. Response Encoding
 - 2.3.1. Response Mode "query.jwt"
 - 2.3.2. Response Mode "fragment.jwt"
 - 2.3.3. Response Mode "form_post.jwt"
 - 2.3.4. Response Mode "jwt"
 - 2.4. Processing rules
3. Client Metadata
4. Authorization Server Metadata
5. Security Considerations
 - 5.1. DoS using specially crafted JWTs
 - 5.2. Protocol Run Integrity
 - 5.3. Mix-Up
 - 5.4. Code Leakage
6. Privacy Considerations
7. IANA Considerations

[7.1. OAuth Dynamic Client Registration Metadata Registration](#)

[7.1.1. Registry Contents](#)

[7.2. OAuth Authorization Server Metadata Registration](#)

[7.2.1. Registry Contents](#)

[8. Normative References](#)

[9. Informative References](#)

[Appendix A. Acknowledgements](#)

[Appendix B. Notices](#)

[Authors' Addresses](#)

1. Introduction

This document defines a new JWT-based mode to encode OAuth authorization responses. Clients are enabled to request the transmission of the authorization response parameters along with additional data in JWT format. This mechanism enhances the security of the standard authorization response with support for signing and optional encryption of the response. A signed response provides message integrity, sender authentication, audience restriction, and protection from mix-up attacks. Encrypting the response provides confidentiality of the response parameter values. The JWT authorization response mode can be used in conjunction with any response type.

1.1. Notational Conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

1.2. Terms and definitions

For the purpose of this document, the terms defined in [[RFC6749](#)] and [[OIDC](#)] apply.

1.3. Symbols and Abbreviated terms

API – Application Programming Interface

CSRF - Cross Site Request Forgery

HTTP – Hyper Text Transfer Protocol

OIDF - OpenID Foundation

TLS – Transport Layer Security

2. JWT-based Response Mode

This document defines a new JWT-based [\[RFC7519\]](#) mode to encode OAuth [\[RFC6749\]](#) authorization response parameters. All response parameters defined for a given response type are conveyed in a JWT along with additional claims used to further protect the transmission. Since there are different techniques to encode the JWT itself in the response to the client, namely query URI parameter, fragment component and form post, this document defines a set of response mode values in accordance with [\[OIDM\]](#) corresponding to these techniques.

2.1. The JWT Response Document

The JWT always contains the following data utilized to secure the transmission:

- `iss` - the issuer URL of the authorization server that created the response
- `aud` - the `client_id` of the client the response is intended for
- `exp` - expiration of the JWT. A maximum JWT lifetime of 10 minutes is RECOMMENDED.

The JWT MUST furthermore contain the authorization endpoint response parameters as defined for the particular response types, even in case of an error response. Authorization endpoint response parameter names and string values are included as JSON strings and numerical values (e.g., `expires_in` value) are included as JSON numbers. This pattern is applicable to all response types including those defined in [\[OIDM\]](#). The following subsection illustrates the pattern for the response type "code".

Note: Additional authorization endpoint response parameters defined by extensions, e.g. `session_state` as defined in [\[OISM\]](#), will also be added to the JWT.

The JWT response document MAY contain further element, e.g. the claims defined in [\[RFC7519\]](#). Implementation SHOULD adhere to the respective processing rules and ignore unrecognized elements.

2.1.1. Example Response Type "code"

For the grant type authorization "code" the JWT contains the response parameters as defined in [\[RFC6749\]](#), sections 4.1.2:

- `code` - the authorization code
- `state` - the state value as sent by the client in the authorization request, if the client included a state parameter

The following example shows the JWT claims for a successful "code" authorization response:

```
{
  "iss": "https://accounts.example.com",
  "aud": "s6BhdRkqt3",
  "exp": 1311281970,
  "code": "PyyFaux2o7Q0YfXBU32jhw.5FXSQpvr8akv9CeRDSd0QA",
  "state": "S8NJ7uqk5fY4EjNvP_G_FtyJu6pUsvH9jsYni9dMAJw"
}
```

In case of an error response, the JWT contains the error response parameters as defined in [\[RFC6749\]](#), sections 4.1.2.1:

- The following example shows the JWT payload for such an error response:

```
{
  "iss": "https://accounts.example.com",
  "aud": "s6BhdRkqt3",
  "exp": 1311281970,
  "error": "access_denied",
  "state": "S8NJ7uqk5fY4EjNvP_G_FtyJu6pUsvH9jsYni9dMAJw"
}
```

The JWT is either signed, or signed and encrypted. If the JWT is both signed and encrypted, the JSON document will be signed then encrypted, with the result being a Nested JWT, as defined in [RFC7519].

For guidance on key management in general and especially on use of symmetric algorithms for signing and encrypting based on client secrets see section 10 of [OIDC].

This document defines the following response mode values:

- ### 2.3.1. Response Mode "query.jwt"

This is an example response (line breaks for display purposes only):

[illegible]

Note: "query.jwt" MUST NOT be used in conjunction with response types that contain "token" or "id_token" unless the response JWT is encrypted to prevent token leakage in the URL.

2.3.2. Response Mode "fragment.jwt"

The response mode "fragment.jwt" causes the authorization server to send the authorization response as HTTP redirect to the redirect URI of the client. The authorization server adds the parameter response containing the JWT as defined in [Section 2.1.](#) to the fragment component of the redirect URI using the "application/x-www-form-urlencoded" format.

This is an example response (line breaks for display purposes only):

```
HTTP/1.1 302 Found
Location: https://client.example.com/cb#
response=eyJraWQiOiJsYWViIiwiaWxnIjoiaRVMYNTYifQ.eyJhZG9rZW5fdHlwZSI6ICJiZWZyZXIiLCAGImV4cGlyZXNfaW4iOiAzNjAwLCAGInNjb3BlIjoiaWV4YW1wbGUifQ.g_96IM2t_6Dazm1Jpb2gb02EXe5IKTw2bYS7l9Y1RI8HbNPYN5EdNjxcWeL5LTQaUAZ2PtJoHbTdjMvNa3xbVg
```

2.3.3. Response Mode "form_post.jwt"

The response mode "form_post.jwt" uses the technique described in [\[OIFP\]](#) to convey the JWT to the client. The response parameter containing the JWT is encoded as HTML form value that is auto-submitted in the User Agent, and thus is transmitted via the HTTP POST method to the Client, with the result parameters being encoded in the body using the "application/x-www-form-urlencoded" format.

This is an example response from the authorization server to the user agent (line breaks for display purposes only),

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Cache-Control: no-cache, no-store
Pragma: no-cache

<html>
<head><title>Submit This Form</title></head>
<body onload="javascript:document.forms[0].submit()">
  <form method="post" action="https://client.example.com/cb">
    <input type="hidden" name="response"
      value="eyJraWQiOiJsYWViIiwiaWxnIjoiaRVMYNTYifQ.eyJhZG9rZW5fdHlwZSI6ICJiZWZyZXIiLCAGImV4cGlyZXNfaW4iOiAzNjAwLCAGInNjb3BlIjoiaWV4YW1wbGUifQ.g_96IM2t_6Dazm1Jpb2gb02EXe5IKTw2bYS7l9Y1RI8HbNPYN5EdNjxcWeL5LTQaUAZ2PtJoHbTdjMvNa3xbVg" />
    </form>
  </body>
</html>
```

which results in the following POST request to the client's redirect URI.

2.3.4. Response Mode "jwt"

The response mode "jwt" is a shortcut and indicates the default redirect encoding (query, fragment) for the requested response type. The default for response type "code" is "query.jwt" whereas the default for "token" and the response types defined in [OIDM], except "none", is "fragment.jwt".

2.4. Processing rules

Assumption: the client remembers the authorization server to which it sent the authorization request and binds this information to the user agent.

The client **MUST** process the JWT secured response as follows:

1. (OPTIONAL) The client decrypts the JWT using the default key for the respective issuer or, if applicable, determined by the kid JWT header parameter. The key might be a private key, where the corresponding public key is registered with the expected issuer of the response ("use": "enc" via the client's metadata jwks or jwks_uri) or a key derived from its client secret (see [Section 2.2](#)).
2. The client obtains the iss element from the JWT and checks whether its value is well known and identifies the expected issuer of the authorization process in examination. If the check fails, the client MUST abort processing and refuse the response.
3. The client obtains the aud element from the JWT and checks whether it matches the client id the client used to identify itself in the corresponding authorization request. If the check fails, the client MUST abort processing and refuse the response.
4. The client checks the JWT's exp element to determine if the JWT is still valid. If the check fails, the client MUST abort processing and refuse the response.
5. The client MUST check the signature of the JWT according to [\[RFC7515\]](#) and the algorithm none ("alg": "none") MUST NOT be accepted. If the check fails, the client MUST abort processing and refuse the response.

The client will perform further checks, e.g. for CSRF detection, which are out of scope of this specification. Please see [RFC9700] for more security recommendations.

Note: The way the client obtains the keys for verifying the JWT's signature (step 5) is out of scope of this document. Established mechanism such as [OIDD] or [RFC8414] SHOULD be utilized.

The client MUST NOT process the grant type specific authorization response parameters before all checks succeed.

3. Client Metadata

The Dynamic Client Registration Protocol [RFC7591] defines an API for dynamically registering OAuth 2.0 client metadata with authorization servers. The metadata defined by [RFC7591], and registered extensions to it, also imply a general data model for clients that is useful for authorization server implementations even when the Dynamic Client Registration Protocol isn't in play. Such implementations will typically have some sort of user interface available for managing client configuration.

The following client metadata parameters are introduced by this specification:

- `authorization_signed_response_alg` The JWS [RFC7515] `alg` algorithm REQUIRED for signing authorization responses. If this is specified, the response will be signed using JWS and the configured algorithm. If unspecified, the default algorithm to use for signing authorization responses is RS256. The algorithm `none` is not allowed.
- `authorization_encrypted_response_alg` The JWE [RFC7516] `alg` algorithm REQUIRED for encrypting authorization responses. If both signing and encryption are requested, the response will be signed then encrypted, with the result being a Nested JWT, as defined in JWT [RFC7519]. The default, if omitted, is that no encryption is performed.
- `authorization_encrypted_response_enc` The JWE [RFC7516] `enc` algorithm REQUIRED for encrypting authorization responses. If `authorization_encrypted_response_alg` is specified, the default for this value is A128CBC-HS256. When `authorization_encrypted_response_enc` is included, `authorization_encrypted_response_alg` MUST also be provided.

The `jwtks_uri` or `jwtks` metadata parameters can be used by clients to register their public encryption keys.

4. Authorization Server Metadata

Authorization servers SHOULD publish the supported algorithms for signing and encrypting the JWT of an authorization response by utilizing OAuth 2.0 Authorization Server Metadata [RFC8414] parameters.

The following parameters are introduced by this specification:

- `authorization_signing_alg_values_supported` OPTIONAL. A JSON array containing a list of the JWS [RFC7515] signing algorithms (`alg` values) supported by the authorization endpoint to sign the response.
- `authorization_encryption_alg_values_supported` OPTIONAL. A JSON array containing a list of the JWE [RFC7516] encryption algorithms (`alg` values) supported by the authorization endpoint to encrypt the response.
- `authorization_encryption_enc_values_supported` OPTIONAL. A JSON array containing a list of the JWE [RFC7516] encryption algorithms (`enc` values) supported by the authorization endpoint to encrypt the response.

Authorization servers SHOULD publish the supported response mode values utilizing the parameter `response_modes_supported` as defined in [RFC8414]. This document introduces the following possible values:

- `query.jwt`
- `fragment.jwt`
- `form_post.jwt`
- `jwt`

5. Security Considerations

As JARM is used as a component in OAuth, many of the security considerations listed in OAuth 2.0 Security Best Current Practice [RFC9700] apply. In addition, for the mechanisms described in this document, the following security considerations apply.

5.1. DoS using specially crafted JWTs

JWTs could be crafted to have an issuer that resolves to a JWK set URL with huge content, or is delivered very slowly, consuming server networking bandwidth and compute capacity. The resolved JWK set URL could also be used to DDoS targets on the web.

The client therefore MUST first check that the issuer of the JWT is well-known and expected for the particular authorization response before it uses this data to obtain the key needed to check the JWT's signature.

5.2. Protocol Run Integrity

An OAuth protocol run is made of many distinct message exchanges between the client and server to complete the issuance of access and refresh tokens. Even if every message itself is integrity protected, it is still conceivable that one or more of the messages are exchanged with another message created for a different protocol run. The leakage and reuse of encrypted messages in [Section 5.4](#) is an example of such problems. To mitigate this problem, it is considered good practice to implement additional protection provided by PKCE [RFC7636] as described in [RFC9700].

5.3. Mix-Up

Mix-up is an attack on scenarios where an OAuth client interacts with multiple authorization servers. The goal of the attack is to obtain an authorization code or an access token by tricking the client into sending those credentials to the attacker instead of using them at the respective endpoint at the authorization/resource server.

The JWT secured response mode enables clients to detect this attack by providing an identification of the sender (iss) and the intended audience of the authorization response (aud).

5.4. Code Leakage

Authorization servers MAY encrypt the authorization response therewith providing a means to prevent leakage of authorization codes in the user agent (e.g. during transmission, in browser history or via referrer headers). Note, however, that the entire response is then potentially subject to leakage. An encrypted response doesn't remove the need for additional protections provided by mechanisms such as PKCE [RFC7636].

6. Privacy Considerations

JARM only defines an alternative way of encoding the authorization response message and therefore does not materially impact the privacy considerations of OAuth 2.0, which is a complex and flexible framework with broad-ranging privacy implications.

The content of a conventional authorization response message (e.g., code and state) does not typically convey personally identifiable information (PII). However, using encrypted JARM may improve privacy by reducing the potential for inadvertent information disclosure in cases where the authorization response might contain PII (such as other response types or extensions).

7. IANA Considerations

7.1. OAuth Dynamic Client Registration Metadata Registration

This specification requests registration of the following client metadata definitions in the IANA "OAuth Dynamic Client Registration Metadata" registry established by [\[RFC7591\]](#):

7.1.1. Registry Contents

- Client Metadata Name: `authorization_signed_response_alg`
- Client Metadata Description: String value indicating the client's desired authorization response signing algorithm.
- Change Controller: IESG
- Specification Document(s): [Section 3](#) of this specification
- Client Metadata Name: `authorization_encrypted_response_alg`
- Client Metadata Description: String value specifying the desired authorization response encryption algorithm (alg value).
- Change Controller: IESG
- Specification Document(s): [Section 3](#) of this specification
- Client Metadata Name: `authorization_encrypted_response_enc`
- Client Metadata Description: String value specifying the desired authorization response encryption algorithm (enc value).
- Change Controller: IESG
- Specification Document(s): [Section 3](#) of this specification

7.2. OAuth Authorization Server Metadata Registration

This specification requests registration of the following value in the IANA "OAuth Authorization Server Metadata" registry established by [\[RFC8414\]](#).

7.2.1. Registry Contents

- Metadata Name: `authorization_signing_alg_values_supported`
- Metadata Description: JSON array containing a list of algorithms supported by the authorization server for authorization response signing.
- Change Controller: IESG
- Specification Document(s): [Section 4](#) of this specification
- Metadata Name: `authorization_encryption_alg_values_supported`
- Metadata Description: JSON array containing a list of algorithms supported by the authorization server for authorization response encryption (alg value).
- Change Controller: IESG
- Specification Document(s): [Section 4](#) of this specification
- Metadata Name: `authorization_encryption_enc_values_supported`

- Metadata Description: JSON array containing a list of algorithms supported by the authorization server for authorization response encryption (enc value).
- Change Controller: IESG
- Specification Document(s): [Section 4](#) of this specification

8. Normative References

- [OIDM] de Medeiros, B., Ed., Scurtescu, M., Tarjan, P., and M.B. Jones, "OAuth 2.0 Multiple Response Type Encoding Practices", 25 February 2014, <http://openid.net/specs/oauth-v2-multiple-response-types-1_0.html>.
- [OIFP] Jones, M. and B. Campbell, "OAuth 2.0 Form Post Response Mode", 27 April 2015, <https://openid.net/specs/oauth-v2-form-post-response-mode-1_0.html>.
- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <<https://www.rfc-editor.org/info/rfc6749>>.
- [RFC7515] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Signature (JWS)", RFC 7515, DOI 10.17487/RFC7515, May 2015, <<https://www.rfc-editor.org/info/rfc7515>>.
- [RFC7516] Jones, M. and J. Hildebrand, "JSON Web Encryption (JWE)", RFC 7516, DOI 10.17487/RFC7516, May 2015, <<https://www.rfc-editor.org/info/rfc7516>>.
- [RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <<https://www.rfc-editor.org/info/rfc7519>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.

9. Informative References

- [OIDC] Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1", 8 November 2014, <http://openid.net/specs/openid-connect-core-1_0.html>.
- [OIDD] Sakimura, N., Bradley, J., Jones, M., and E. Jay, "OpenID Connect Discovery 1.0 incorporating errata set 1", 8 November 2014, <https://openid.net/specs/openid-connect-discovery-1_0.html>.
- [OISM] de Medeiros, B., Agarwal, N., Sakimura, N., Bradley, J., and M.B. Jones, "OpenID Connect Session Management 1.0", 14 March 2022, <http://openid.net/specs/openid-connect-session-1_0.html>.
- [RFC7591] Richer, J., Ed., Jones, M., Bradley, J., Machulak, M., and P. Hunt, "OAuth 2.0 Dynamic Client Registration Protocol", RFC 7591, DOI 10.17487/RFC7591, July 2015, <<https://www.rfc-editor.org/info/rfc7591>>.
- [RFC7636] Sakimura, N., Ed., Bradley, J., and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients", RFC 7636, DOI 10.17487/RFC7636, September 2015, <<https://www.rfc-editor.org/info/rfc7636>>.

[RFC8414] Jones, M., Sakimura, N., and J. Bradley, "OAuth 2.0 Authorization Server Metadata", RFC 8414, DOI 10.17487/RFC8414, June 2018, <<https://www.rfc-editor.org/info/rfc8414>>.

[RFC9700] Lodderstedt, T., Bradley, J., Labunets, A., and D. Fett, "Best Current Practice for OAuth 2.0 Security", BCP 240, RFC 9700, DOI 10.17487/RFC9700, January 2025, <<https://www.rfc-editor.org/info/rfc9700>>.

Appendix A. Acknowledgements

The following people contributed to this document:

- Torsten Lodderstedt (SPRIND), Editor
- Brian Campbell (Ping Identity), Co-editor
- Nat Sakimura (NAT Consulting LLC) -- Chair
- Dave Tonge (Moneyhub) -- Chair
- Joseph Heenan (Authlete)
- Ralph Bragg (Raidiam)
- Vladimir Dzhuvinov (Connect2ID)
- Michael Schwartz (Gluu)
- Filip Skokan (Okta)

Appendix B. Notices

Copyright (c) 2025 The OpenID Foundation.

The OpenID Foundation (OIDF) grants to any Contributor, developer, implementer, or other interested party a non-exclusive, royalty free, worldwide copyright license to reproduce, prepare derivative works from, distribute, perform and display, this Implementers Draft, Final Specification, or Final Specification Incorporating Errata Corrections solely for the purposes of (i) developing specifications, and (ii) implementing Implementers Drafts, Final Specifications, and Final Specification Incorporating Errata Corrections based on such documents, provided that attribution be made to the OIDF as the source of the material, but that such attribution does not indicate an endorsement by the OIDF.

The technology described in this specification was made available from contributions from various sources, including members of the OpenID Foundation and others. Although the OpenID Foundation has taken steps to help ensure that the technology is available for distribution, it takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this specification or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any independent effort to identify any such rights. The OpenID Foundation and the contributors to this specification make no (and hereby expressly disclaim any) warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to this specification, and the entire risk as to implementing this specification is assumed by the implementer. The OpenID Intellectual Property Rights policy (found at openid.net) requires contributors to offer a patent promise not to assert certain patent claims against other contributors and against implementers. OpenID invites any interested party to bring to its attention any copyrights, patents, patent applications, or other proprietary rights that may cover technology that may be required to practice this specification.

Authors' Addresses

Torsten Lodderstedt

SPRIND

Email: torsten@lodderstedt.net

Brian Campbell

Ping Identity

Email: bcampbell@pingidentity.com